

Project 3: Reinforcement Learning

(Thanks to John DeNero and Dan Klein!)

Deadline: Monday, November 17th, 2025 (before class for 001 and end of the day for D01).

Please review the "What to Submit" section before submitting.



Pacman seeks reward.
Should he eat or should he run?
When in doubt, Q-learn.

Introduction

In this project, you will implement Q-learning. You will test your agents first on Gridworld (from class), then apply them to a simulated robot controller (Crawler) and Pacman. You can download the code from Canvas. The code for this project contains the following files:

Files you will edit:

qlearningAgents.py	Q-learning agents for Gridworld, Crawler and Pacman.
analysis.py	A file to put your answers to questions given in the project.

Files you should read but NOT edit

valueIterationAgents.py	A value iteration agent for solving known MDPs.
mdp.py	Defines methods on general MDPs.
learningAgents.py	Defines the base classes ValueEstimationAgent and QLearningAgent, which your agents will extend.
util.py	Utilities, including util.Counter, which is particularly useful for Q-learners.
gridworld.py	The Gridworld implementation.
featureExtractors.py	Classes for extracting features on (state,action) pairs. Used for the approximate Q-learning agent (in qlearningAgents.py).

Files you can ignore

environment.py	Abstract class for general reinforcement learning environments. Used by gridworld.py.
graphicsGridworldDisplay.py	Gridworld graphical display.

graphicsUtils.py	Graphics utilities.
textGridworldDisplay.py	Plug-in for the Gridworld text interface.
crawler.py	The crawler code and test harness. You will run this but not edit it.
graphicsCrawlerDisplay.py	GUI for the crawler robot.

Evaluation: Your code will be autograded for technical correctness.

Please do not change the names of any provided functions or classes within the code, or you will wreak havoc on the autograder. If your code works correctly on one or two of the provided examples but doesn't get full credit from the autograder, you most likely have a subtle bug that breaks one of our more thorough test cases; you will need to debug more fully by reasoning about your code and trying small examples of your own. That said, bugs in the autograder are not impossible, so please do contact the staff if you believe that there has been an error in the grading.

Academic Integrity: We will be checking your code against other submissions in the class for logical redundancy. If you copy someone else's code and submit it with minor changes, we will know. These cheat detectors are quite hard to fool, so please don't try. We trust you all to submit your own work only; please don't let us down. If you do, we will pursue the strongest consequences available to us.

Getting Help: You are not alone! If you find yourself stuck on something, contact the course staff for help. Office hours, section, and Piazza are there for your support; please use them. If you can't make our office hours, let us know and we will schedule more. We want these projects to be rewarding and instructional, not frustrating and demoralizing. But, we don't know when or how to help unless you ask.

Q-learning

Question 1: Q-learning (15 points)

You will now write a Q-learning agent, which does very little on construction, but instead learns by trial and error from interactions with the environment through its update (state, action, nextState, reward) method. A stub of a Q-learner is specified in `QLearningAgent` in `qlearningAgents.py`, and you can select it with the option '-a q'. For this question, you must implement the `update`, `getValue`, `getQValue`, and `getPolicy` methods.

Note: For `getPolicy`, you should break ties randomly for better behavior. The `random.choice()` function will help. In a particular state, actions that your agent hasn't seen before still have a Q-value, specifically a Q-value of zero, and if all of the actions that your agent has seen before have a negative Q-value, an unseen action may be optimal.

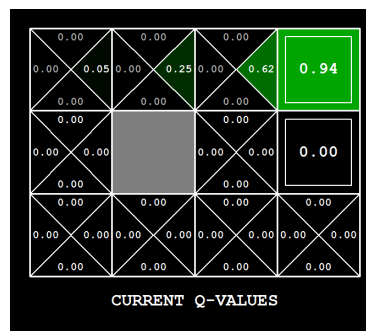
Important: Make sure that in your `getValue` and `getPolicy` functions, you only access Q values by calling `getQValue`. This abstraction will be helpful for questions 4 or 5, when you override `getQValue` to use features of state-action pairs rather than state-action pairs directly.

With the Q-learning update in place, you can watch your Q-learner learn under manual control, using the keyboard:

```
python gridworld.py -a q -k 5 -m
```

The -k parameter controls the number of episodes your agent learns from. Watch how the agent learns about the state it was just in, not the one it moves to, and "leaves learning in its wake."

Hint: to help with debugging, you can turn off noise by using the `--noise 0.0` parameter (though this obviously makes Q-learning less interesting). If you manually steer Pacman north and then east along the optimal path for four episodes, you should see the following Q-values:



Grading: We will run your Q-learning agent on an example of our own and check that it learns the same Q-values and policy as our reference implementation when each is presented with the same set of examples.

Question 2: Epsilon Greedy (10 points)

Complete your Q-learning agent by implementing epsilon-greedy action selection in `getAction`, meaning it chooses random actions an epsilon fraction of the time, and follows its current best Q-values otherwise.

```
python gridworld.py -a q -k 100
```

Your final Q-values should resemble those of your value iteration agent, especially along well-traveled paths. However, your average returns will be lower than the Q-values predict because of the random actions and the initial learning phase.

You can choose an element from a list uniformly at random by calling the `random.choice` function. You can simulate a binary variable with probability p of success by using `util.flipCoin(p)`, which returns `True` with probability p and `False` with probability $1-p$.

Question 3: Bridge Crossing Revisited (5 points)

With no additional code, you should now be able to run a Q-learning crawler robot:

```
python crawler.py
```

If this doesn't work, you've probably written some code too specific to the GridWorld problem and you should make it more general to all MDPs.

This will invoke the crawling robot from class using your Q-learner. Play around with the various learning parameters to see how they affect the agent's policies and actions. Note that the step delay is a parameter of the simulation, whereas the learning rate and epsilon are parameters of your learning algorithm, and the discount factor is a property of the environment.

Grading: We give you a point for free here, but play around with the crawler anyway!

Approximate Q-learning and State Abstraction

Question 4 (15 points)

Implement an approximate Q-learning agent that learns weights for features of states, where many states might share the same features. Write your implementation in `ApproximateQAgent` class in `qlearningAgents.py`, which is a subclass of `PacmanQAgent`.

Note: Approximate Q-learning assumes the existence of a feature function $f(s,a)$ over state and action pairs, which yields a vector $f_1(s,a) \dots f_i(s,a) \dots f_n(s,a)$ of feature values. We provide feature functions for you in `featureExtractors.py`. Feature vectors are `util.Counter` (like a dictionary) objects containing the non-zero pairs of features and values; all omitted features have value zero.

The approximate Q-function takes the following form

$$Q(s, a) = \sum_i^n f_i(s, a) w_i$$

where each weight w_i is associated with a particular feature $f_i(s,a)$. In your code, you should implement the weight vector as a dictionary mapping features (which the feature extractors will return) to weight values. You will update your weight vectors similarly to how you updated Q-values:

$$w_i \leftarrow w_i + \alpha [\textit{correction}] f_i(s, a)$$
$$\textit{correction} = (R(s, a) + \gamma V(s')) - Q(s, a)$$

Note that the correction term is the same as in normal Q-learning.

By default, `ApproximateQAgent` uses the `IdentityExtractor`, which assigns a single feature to every (state,action) pair. With this feature extractor, your approximate Q-learning agent should work identically to `PacmanQAgent`. You can test this with the following command:

```
python pacman.py -p ApproximateQAgent -x 2000 -n 2010 -l smallGrid
```

Important: `ApproximateQAgent` is a subclass of `QLearningAgent`, and it therefore shares several methods like `getAction`. Make sure that your methods in `QLearningAgent` call `getQValue` instead of accessing Q-values directly, so that when you override `getQValue` in your approximate agent, the new approximate q-values are used to compute actions.

Once you're confident that your approximate learner works correctly with the identity features, run your approximate Q-learning agent with our custom feature extractor, which can learn to win with ease:

```
python pacman.py -p ApproximateQAgent -a extractor=SimpleExtractor -x 50 -n 60 -l mediumGrid
```

Even much larger layouts should be no problem for your ApproximateQAgent. (warning: this may take a few minutes to train)

```
python pacman.py -p ApproximateQAgent -a extractor=SimpleExtractor -x 50 -n 60 -l mediumClassic
```

If you have no errors, your approximate Q-learning agent should win almost every time with these simple features, even with only 50 training games.

Grading: We will run your agent on the mediumGrid layout 100 times using a fixed random seed. You will receive 1 point if your agent wins more than 25% of its games, 2 points if it wins more than 50% of its games, and 3 points if it wins more than 75% of its games. You can try your agent out under these conditions with

```
python pacman.py -p ApproximateQAgent -a extractor=SimpleExtractor -x 50 -n 150 -l mediumGrid -q -f
```

Congratulations! You have a learning Pacman agent!

What to submit:

qlearningAgents.py and qlearningAgents.txt and partner.txt

You will fill in portions of qlearningAgents.py during the assignment. You should submit these files with your code and comments to Canvas. Don't forget your txt version of these files. Please *do not* change the other files in this distribution or submit any of our original files other than the ones required. *If you work with a partner, please write their name in a file named partner.txt*